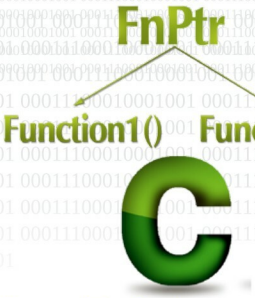

Function Pointers and Callbacks An Odyssey

Function pointers are among the most powerful tools in C, but are a bit of a pain during the initial stages of learning. This article demonstrates the basics of function pointers, and how to use them to implement function callbacks in C. C++ takes a slightly different route for callbacks, which is another journey altogether.

A pointer is a special kind of variable that holds the address of another variable. The same concept applies to function pointers, except that instead of pointing to variables, they point to functions. If you declare an array, say, *int a[10];* then the array name *'a'* will in most contexts (in an expression or passed as a function parameter) 'decay' to a non-modifiable pointer to its first element (even though pointers and arrays are not equivalent while declaring/defining them, or when used as operands of the *sizeof* operator). In the same way, for *int func();*, *'func'* decays to a non-modifiable pointer to a function. You can think of *'func'* as a *const* pointer for the time being.

But can we declare a non-constant pointer to a function? Yes, we can—just like we declare a non-constant pointer to a variable:

```
int (*ptrFunc) ();
```

Here, *ptrFunc* is a pointer to a function that takes no arguments and returns an integer. DO NOT forget to put in the parenthesis, otherwise the compiler will assume that *ptrFunc* is a normal function name, which takes nothing and returns a pointer to an integer.

Let's try some code. Check out the following simple program:

```
#include<stdio.h>

/* function prototype */
int func(int, int);
int main(void)
{
    int result;
    /* calling a function named func */
    result = func(10,20);
    printf("result = %d\n",result);
    return 0;
}

/* func definition goes here */
int func(int x, int y)
{
return x+y;
}
```

As expected, when we compile it with *gcc -g -o example1 example1.c* and invoke it with *./example1*, the output is as follows:

```
result = 30
```